

MOATLY

Moat App — Phase 2 Technical Breakdown

The Moats themselves: Megolm group messaging via vodozemac, the seven-day deletion client, Major Statements, the complaint flow, soft-auth as a time-bounded stand-in.

Prepared for: Engineering lead, Moatly

Document version: 0.1 · Status: Draft for review

Date: April 2026

0. How to read this document

This is the second of three technical breakdowns for the Moat mobile app. *The first* (v0.1) covered Phase 1 — the app-only MVP, without Moats. *This document* covers Phase 2: the introduction of Moats themselves.

Phase 2 is the phase in which the app stops being a journaling shell and becomes the product described in the Build Brief. It introduces the **Moat service client** (the mobile side of Megolm group messaging via vodozemacs), the **seven-day deletion client** (the on-device half of the subsystem specified in §9 of the Build Brief), the **Major Statement** composition and display, the **complaint and reassignment flow**, and a time-bounded **soft-auth** that exists only until the device ships in Phase 3.

The document structure mirrors v0.1. Sections 1–3 are decisions, architecture deltas, and the updated module catalogue. Sections 4–6 are the per-module specifications. Sections 7–9 are cross-cutting concerns (offline behaviour at scale, cycle boundaries, session rotation), data flows, and the test strategy. Sections 10–12 are the implementation plan, open questions, and what Phase 3 will cover.

0.1 What this document assumes

- The reader has read the Build Brief v1.0, and in particular §2 (non-negotiable principles), §5.4 (the Moat service), §5.10 (complaint flow), §6.2 (Olm/Megolm via vodozemacs), §7 (data model), §8 (API surface), and §9 (the seven-day deletion system).
- The reader has read the Phase 1 breakdown (v0.1). Modules specified there (crypto.seed, crypto.keyring, infra.db, infra.secure-store, domain.onboarding, ui.covenant, domain.compass, domain.rings, domain.linkage, ui.lamp, infra.logging, infra.notifications) are referenced freely.
- Decisions ratified at the end of Phase 1 (framework, state, local DB, testing) are in force.

0.2 What this document does not cover

- The backend Go implementation of the Moat service. Covered by a separate backend breakdown.
- The seven-day deletion pipeline as it runs server-side. Covered by the backend breakdown and §9 of the Build Brief.
- BLE, firmware, glow, combination entry, composite identity code. That is Phase 3.
- The Unity ritual app.

0.3 Document map

§	Contents	Primary reader
1	Decisions required before Week 1 of Phase 2.	Lead, founder
2	Architecture deltas from Phase 1 — what changes, what doesn't.	All engineers
3	The Phase 2 module catalogue.	Lead, mobile leads
4	The vodozemacs integration — FFI, session	Mobile + security

§	Contents	Primary reader
	lifecycle, error model.	
5	Per-module specifications (Phase 2 modules).	Mobile engineers
6	The seven-day deletion client — the on-device half of §9 of the brief.	Mobile + security
7	Cross-cutting: cycle boundaries, session rotation, soft-auth lifecycle.	All
8	Data flows for the Phase 2 scenarios.	Mobile + QA
9	Test strategy — the adversarial tests that must exist before exit.	QA, lead
10	Implementation plan mapped to the Build Brief's Phase 2 exit gate.	Lead, PM
11	Open questions for decision this week.	Lead, founder
12	What Phase 3 will cover.	Lead

1. Decisions required before Week 1 of Phase 2

Six decisions, each blocking a non-trivial slice of work. Each states a default recommendation; lead ratifies or overrules in writing before work starts.

1.1 vodozemacs FFI binding strategy

Recommendation: UniFFI-generated bindings (uniffi-bindgen-react-native), with a hand-written TypeScript wrapper around the generated surface.

Three paths existed: (a) UniFFI-generated bindings, (b) cxx - rs or a hand-written React Native Turbo Module wrapping vodozemacs directly, (c) compile vodozemacs to WebAssembly and run it in a JavaScript runtime in-app.

Path (c) is rejected — running the crypto inside JS defeats most of the reason we picked a Rust crate in the first place. Path (b) is feasible but the maintenance burden is high: every vodozemacs update requires hand-fixing the binding. Path (a) is how the Element/Matrix ecosystem consumes vodozemacs today; UniFFI generates bindings for several languages, and the React Native adapter (maintained by the community at uniffi-bindgen-react-native) has sufficient momentum that it is a reasonable bet.

The hand-written TypeScript wrapper matters. The UniFFI-generated interface is *ergonomic-but-leaky* — errors come through as untyped strings, handles are raw integers. We wrap it in a module (§5.1) that presents typed errors, branded handles, and a surface that the rest of the app can use without learning FFI semantics.

1.2 Message framing format

Recommendation: Protobuf for the outer envelope; a deliberately small CBOR-like JSON for the encrypted body.

The outer envelope is what the server sees — message type, sender pseudonym index, timestamp, attached Megolm ciphertext. It is Protobuf because it goes on the wire and will be touched by two language runtimes (Go on the server, TS via UniFFI on the client); Protobuf's cross-language tooling is reliable and its schema can be version-gated.

The inner (encrypted) body is JSON. We considered binary CBOR or MsgPack to save a few bytes per message; the savings are trivial at our message volume, and JSON's grep-ability is a real debugging asset in a system where we are forbidden from logging the content.

1.3 WebSocket transport — library choice

Recommendation: The platform-native WebSocket in React Native (which is a thin wrapper over iOS NSURLSessionWebSocketTask and Android OkHttp WebSocket), with a hand-written reconnection layer.

Third-party libraries (socket.io-client, centrifuge, ably-js) all bring features we do not want: auto-reconnect strategies that retry indefinitely, observability hooks that send to their own cloud, transport fallbacks to long-poll. Our WebSocket semantics are simple — authenticated, single-stream, fail-fast — and a 150-line hand-written reconstructor covers it.

1.4 Cycle-boundary clock — device, server, or both

Recommendation: Server authoritative. Device compares against a trusted-time service at cycle-ends-at approach, never drives cycle transitions on its own.

Every Moat carries a `cycle_ends_at` timestamp set by the server. The client shows a countdown and, near zero, asks the server for the new session's keying material. The client does not unilaterally rotate sessions — that is the server's job via POST `/v1/moats/current/session/reset` (per §8.3 of the brief).

If the device clock is wrong by more than ± 5 minutes at the cycle boundary, we surface a soft warning and ask the user to check their clock. We do not enforce a corrective sync — that would leak an identifying bit (the device's time-drift pattern).

1.5 Pseudonym allocation

Recommendation: Server allocates pseudonyms from a curated word-pair list at Moat formation. Pseudonyms rotate at cycle boundary.

Names like 'Obsidian', 'Crane', 'Juniper'. A curated list of ~500 evocative, short, non-offensive words. The server picks one per user per Moat session — pseudonyms rotate every seven days so that cross-cycle identity cannot be reconstructed by another member. The user sees their own pseudonym prominently; they do not choose it. This deliberately prevents the 'screen-name performance' that usernames invite.

1.6 Soft-auth storage and decommission trigger

Recommendation: Soft-auth tokens live in secure store with an explicit `expires_at` field set to the Phase 3 go-live date, not a sliding expiry.

Soft-auth is a stand-in for device auth — email + seed phrase, with the seed phrase doing triple duty (§4.2 of v0.1). We want soft-auth to *vanish* cleanly when the device ships in Phase 3 rather than persisting as a fallback that could become a permanent side-door. Putting a hard expiry on the token forces the decommission to actually happen. Section §7.3 covers the migration.

1.7 Summary table

Decision	Default recommendation	Owner
vodozemac FFI strategy	UniFFI + TS wrapper	Lead + security
Message framing	Protobuf outer + JSON inner	Lead
WebSocket library	Native WebSocket + hand-written reconnect	Lead
Cycle-boundary authority	Server authoritative	Lead
Pseudonym allocation	Server-assigned from a curated list	Product + lead
Soft-auth decommission	Hard expiry tied to Phase 3 go-live	Lead + security

2. Architecture deltas from Phase 1

The four-layer architecture from v0.1 §2.1 is unchanged. What changes is which modules exist at each layer.

2.1 Updated directory layout

Phase 2 adds the following directories (existing Phase 1 structure elided):

```
src/
├── crypto/
│   ├── (Phase 1 modules, unchanged)
│   └── vodozemac/
│       ├── native/                # UniFFI-generated bindings
│       │   ├── ios/
│       │   └── android/
│       ├── index.ts               # TS wrapper around the UniFFI surface
│       ├── errors.ts             # typed error model
│       └── __tests__/             # property tests against test vectors
├── infra/
│   ├── (Phase 1 modules, unchanged)
│   ├── api/
│   │   ├── client.ts             # REST client (auth, accounts, linkage)
│   │   ├── moat.ts               # Moat-specific REST endpoints
│   │   └── schemas/              # Zod schemas for responses
│   ├── ws/
│   │   ├── gateway.ts           # WebSocket connection + reconnect
│   │   ├── heartbeat.ts
│   │   └── frame.ts              # Protobuf envelope decode
│   └── auth/
│       ├── soft-auth.ts          # Phase 2 only; removed in Phase 3
│       └── jwt.ts                # JWT storage + refresh
├── domain/
│   ├── (Phase 1 modules, unchanged)
│   ├── moat/
│   │   ├── session.ts            # Megolm session lifecycle
│   │   ├── membership.ts
│   │   ├── deletion-client.ts    # on-device 7-day deletion
│   │   ├── messages.ts           # message store, dedupe, ordering
│   │   └── major-statement.ts     # the 6-field schema, validation
│   ├── complaints/
│   │   ├── compose.ts            # encrypt a complaint envelope
│   │   └── thresholds.ts         # display logic only; server enforces
│   └── cycle/
│       └── boundary.ts            # cycle clock, countdown, rotation trigger
└── ui/
    ├── (Phase 1 modules, unchanged)
    ├── moat/
    │   ├── home.tsx              # chronological feed
    │   ├── composer.tsx          # whisper / major-statement
    │   ├── major-statement-card.tsx
    │   ├── witness-thread.tsx
    │   └── cycle-countdown.tsx
```

```

├── pseudonym-badge.tsx
├── presence-indicator.tsx
├── complaint/
│   ├── file.tsx
│   └── my-filed.tsx

```

2.2 Module catalogue — Phase 2 scope

Module	Layer	Responsibility	Section
crypto.vodozemac	1	UniFFI boundary, typed wrapper, error model.	§4 + §5.1
infra.auth.jwt	2	JWT storage, refresh, attach to requests.	§5.2
infra.auth.soft-auth	2	Email + seed soft-auth. Time-bounded.	§5.3
infra.api.moat	2	REST endpoints for Moat + complaint flow.	§5.4
infra.ws.gateway	2	WebSocket stream, reconnect, heartbeat.	§5.5
domain.moat.session	3	Megolm session lifecycle, rotation at cycle-end.	§5.6
domain.moat.messages	3	Message store, ordering, dedupe, local index.	§5.7
domain.moat.major-statement	3	Six-field schema, validation, composition.	§5.8
domain.moat.deletion-client	3	On-device seven-day deletion enforcement.	§6
domain.complaints	3	File, track, and display user's own complaints.	§5.9
domain.cycle.boundary	3	Cycle countdown; trigger session rotation.	§5.10
ui.moat.*	4	Chronological feed, composer, cycle countdown UI.	§5.11
ui.complaint.*	4	File a complaint; review 'my filed complaints'.	§5.12

2.3 What Phase 2 intentionally still leaves stubbed

- No BLE, no device, no glow. All Phase 3.
- No combination sequence, no composite identity code. Soft-auth stands in.
- No attachments. Text only. (See §14.3 of the Build Brief — the rule is 'text-only at launch'.)
- No cross-Moat bridges. Moats are islands, always.

3. Catalogue discussion — what each group of modules is for

Before the per-module specifications (§4 onward), a short tour of the catalogue. Each subsection below explains the role of one cluster of modules, the invariant it enforces, and the adversary it defeats.

3.1 The crypto cluster — `crypto.vodozemacs`

This is the only new module at Layer 1. It is the gate between the TypeScript world and the Rust crypto crate. Everything that encrypts, decrypts, or touches Megolm key material crosses this gate.

The invariant: nobody above Layer 2 ever sees a raw Megolm key, session ID, or ciphertext byte array. They see branded handles and typed message objects. This keeps the attack surface for post-audit regressions small.

3.2 The auth cluster — `infra.auth.jwt` and `infra.auth.soft-auth`

Two modules. JWT handling is the permanent auth plumbing — it will still exist in Phase 3, with the JWT issuer changed from 'soft-auth exchange' to 'device-attested challenge exchange'.

Soft-auth is the disposable one. It uses email + seed phrase to produce a device-like token. The brief is explicit that soft-auth 'is a temporary authentication method and is REMOVED in Phase 3' (§10.3). §7.3 of this document specifies how removal works.

3.3 The transport cluster — `infra.api.moat` and `infra.ws.gateway`

REST for everything except real-time message delivery. WebSocket for the stream of new messages as they arrive in a user's current Moat. The WebSocket is authenticated with a short-lived ticket obtained from a REST endpoint rather than the JWT directly, so that the long-lived JWT never has to go through the URL or the Sec-WebSocket-Protocol header.

3.4 The Moat domain cluster — `session`, `messages`, `major-statement`

Three modules that together are the client-side of the Moat service.

- `domain.moat.session` owns the Megolm session lifecycle. Creating sessions, rotating them, dropping old ones, responding to membership changes.
- `domain.moat.messages` is the on-device inbox. It ingests ciphertext from the WebSocket, decrypts via the session module, writes the cleartext to SQLCipher for the seven-day window, and exposes it to the UI.
- `domain.moat.major-statement` is the schema and validation for the six-field structured message. Separate module because it has non-trivial rules (once-per-cycle, required fields, enum constraints).

3.5 The deletion cluster — `deletion-client`

A single module that gets its own section (§6) because it is how we keep the Build Brief's most load-bearing promise on the client side. Three independent mechanisms, just as on the server: local row expiry, Megolm session rotation, SQLCipher key rotation for backups.

3.6 The complaint cluster

The complaint flow is interesting because it is a user-facing feature that the server deliberately does not let the client influence beyond filing. Thresholds (§5.10.2 of the brief) are entirely server-side. The client composes encrypted complaint envelopes, lists the user's own filed complaints for transparency, and displays the 'you have been re-seeded' state if it arrives. The client shows no threshold UI, no indicator that a target is approaching reassignment — because the client does not know.

3.7 The cycle cluster — `domain.cycle.boundary`

A small but load-bearing module. It owns the countdown clock, detects cycle-end, coordinates with the session module to rotate the Megolm session, triggers the deletion pass, and kicks the rings ledger to compute the week's accrual. All the events that happen at cycle-end get their trigger from one place, which makes the 'did it all fire?' test tractable.

4. The vodozemacs integration

This section gets its own chapter because the FFI is the place where *all* the hard questions cluster: memory management across a language boundary, error semantics, how we persist session state, and what the test matrix looks like.

4.1 Why vodozemacs, restated

The Build Brief picked vodozemacs for licensing (Apache 2.0, §6.2), maturity (nine years in Matrix, audited by Least Authority in 2022), and library ecosystem (compared against a still-maturing MLS ecosystem). This breakdown restates that choice only to flag: **we do not revisit it in Phase 2**. If vodozemacs ships a new major version that breaks our pinned API, we take the patch-and-audit path, not the switch-library path.

4.2 The FFI boundary

UniFFI generates bindings from a `.udl` interface file. We write a thin wrapper crate in Rust (`crates/moatly-vodozemacs-ffi`) that:

- Re-exports only the vodozemacs operations we actually use.
- Translates Rust Result types into UniFFI-friendly error enums (so errors arrive on the TS side as discriminated unions, not stringly-typed).
- Serialises session state to a byte array on the Rust side before returning to TS (we never hold a live vodozemacs object across async JS boundaries).
- Zeroises any buffers containing key material before they cross the FFI out.

4.3 What the wrapper exposes

```
// src/crypto/vodozemacs/index.ts – the TS surface above UniFFI.

// Opaque handles – values here are never meaningful outside this module.
export type AccountHandle = { __brand: 'AccountHandle'; handle: string };
export type InboundSession = { __brand: 'InboundSession'; serialized: Uint8Array };
export type OutboundSession = { __brand: 'OutboundSession'; serialized: Uint8Array };

// — 1-to-1 (Olm) – used only for key exchange, not for Moat messages —
export async function createOlmAccount(seed: Uint8Array): Promise<AccountHandle>;
export async function serializeOlmAccount(a: AccountHandle): Promise<Uint8Array>;

// — Group (Megolm) – the hot path —
export async function createOutboundGroupSession(): Promise<OutboundSession>;
export async function encryptGroupMessage(
  session: OutboundSession,
  plaintext: Uint8Array
): Promise<{ ciphertext: Uint8Array; session: OutboundSession }>;

export async function createInboundGroupSession(
  sessionKey: Uint8Array // received via Olm from the sender
): Promise<InboundSession>;

export async function decryptGroupMessage(
  session: InboundSession,
```

```

    ciphertext: Uint8Array
  ): Promise<{ plaintext: Uint8Array; session: InboundSession; messageIndex: number }>;

export async function exportInboundGroupSession(
  session: InboundSession
): Promise<Uint8Array>; // for backup to SQLCipher

export async function importInboundGroupSession(
  exported: Uint8Array
): Promise<InboundSession>;

// — Session discard – explicit zeroisation —
export async function discardSession(
  s: InboundSession | OutboundSession
): Promise<void>;

```

The interface returns a new session object from every mutating operation rather than mutating in place. This is because each `InboundSession` / `OutboundSession` carries forward the advanced ratchet state; the previous serialized bytes are no longer decryptable. It matches how vodozamac thinks about sessions and makes the 'rotate and drop the old one' pattern natural.

4.4 Typed error model

```

// src/crypto/vodozamac/errors.ts

export type VodozamacError =
  | { kind: 'decryption-failed'; reason: string } // generic failure
  | { kind: 'unknown-message-index' } // msg from before session start
  | { kind: 'replay' } // message index reused
  | { kind: 'session-expired' } // session past its lifetime
  | { kind: 'wrong-session-id' } // ciphertext bound to a
different session
  | { kind: 'malformed-ciphertext' } // not parseable as Megolm
  | { kind: 'internal'; detail: string }; // library bug, reproduce

// Every FFI call resolves or throws one of the above.
// No 'Error with string message' ever reaches Layer 3.

```

Typed errors matter here because different failure modes get different UX. A 'replay' error is a silent drop (the message was already shown); a 'malformed-ciphertext' is a hard fail we log as an operational metric; an 'unknown-message-index' is expected when joining mid-cycle and rendered as 'message from before you joined, not available'.

4.5 Session persistence

Megolm sessions persist for the duration of a cycle. We serialize them after every state change and write to a SQLCipher table:

```

CREATE TABLE megolm_sessions (
  session_id TEXT NOT NULL,          -- Megolm session identifier
  moat_id    TEXT NOT NULL,
  direction  TEXT NOT NULL CHECK (direction IN ('inbound', 'outbound')),
  cycle_number INTEGER NOT NULL,
  serialized BLOB NOT NULL,         -- exported session state
  created_at INTEGER NOT NULL,

```

```

discarded_at      INTEGER,                -- non-null once dropped
PRIMARY KEY (session_id, direction)
);
CREATE INDEX idx_megolm_cycle ON megolm_sessions (moat_id, cycle_number);

```

-- This table is encrypted at rest (SQLCipher). Even so, the serialized
 -- column never leaves the app, never appears in a log, never hits the
 -- network.

On cycle rotation, sessions from the previous cycle are wiped via DELETE + VACUUM on this table. This is one of the three deletion mechanisms on the client (§6).

4.6 Test matrix for the FFI

Test	What it proves
Round-trip test vectors	Encrypt(plaintext, session) → decrypt(ciphertext, session) yields original plaintext for 1,000 synthetic cases.
Message-index invariants	decryptGroupMessage returns strictly increasing messageIndex values. Out-of-order deliveries are accepted up to a window; beyond it, { kind: 'unknown-message-index' }.
Replay rejection	Decrypting the same ciphertext twice yields { kind: 'replay' } on the second call.
Cross-platform parity	A session serialized on iOS loads on Android, and vice versa. Avoids subtle endianness bugs in the wrapper crate.
Memory pressure	After 10,000 encrypt calls and 10,000 decrypt calls, the process RSS is stable within 10MB. Catches FFI leaks early.
Zeroisation	After discardSession, the previous serialized bytes in memory are overwritten (verified via a stub injected into the wrapper crate).

5. Per-module specifications

Same template as v0.1: Responsibility, Public interface, Data, Dependencies, Notes, Risks, Acceptance criteria.

5.1 `crypto.vodozemacs` — the TS wrapper

Responsibility

Owens the FFI described in §4. Exposes a branded-handle, typed-error interface to the rest of the app. Holds no state of its own between calls — every session is passed in and returned out explicitly, so that Layer 3 can decide where and how to persist.

Public interface

As specified in §4.3. The wrapper is a thin adapter over UniFFI-generated bindings and adds:

- Branded types, so that a TS reviewer cannot accidentally pass an outbound session where an inbound one is required.
- Error translation from UniFFI's generated enum to the module's typed `VodozemacsError`.
- Explicit zeroisation on `discardSession` — calls into the wrapper crate's `zeroise_buffer`, does not rely on GC.

Data

None persisted here. All persistence lives in `domain.moat.session` via the `megolm_sessions` table.

Dependencies

- The UniFFI-generated binding module.
- The wrapper Rust crate (`crates/moatly-vodozemacs-ffi`).

Notes

- An ESLint rule blocks any imports from this module outside `domain.moat.*` and its tests. The rest of the app has no reason to touch it.
- Pinning: `vodozemacs` is pinned to an exact semver. Upgrades go through the security engineer and an external audit re-run on the wrapper.

Risks

- UniFFI React Native support is community-maintained. If the community project goes unmaintained, the fallback path is a hand-written Turbo Module — budget two weeks for that if we ever need to do it.
- Async callbacks across the JS bridge have a measurable cost. We batch decrypts when ingesting a backlog (e.g. rejoining after a long offline stretch) to keep that cost linear rather than per-message.

Acceptance criteria

- 100% branch coverage on `errors.ts` — every UniFFI error variant maps to exactly one `VodozemacsError`.
- The test matrix in §4.6 passes on iOS and Android CI on every commit.

- A burn-in test running the encrypt/decrypt loop for 24 hours shows no RSS growth beyond 10MB.

5.2 infra.auth.jwt — the permanent auth plumbing

Responsibility

Store the current JWT in secure store, refresh it before expiry, attach it to API requests and WebSocket ticket requests. Also: on 401 from the server, trigger a forced re-authentication via whatever auth path is current (soft-auth in Phase 2, device in Phase 3) — this module does not know which.

Public interface

```
export async function currentJwt(): Promise<string | null>;

export async function setJwt(jwt: string, expiresAt: number): Promise<void>;

export async function clearJwt(): Promise<void>;

export async function ensureFreshJwt(): Promise<string>;
  // refreshes if <2 min to expiry; returns the live JWT.
  // throws AuthRequiredError if no refresh path is available
  // (caller then invokes the current auth path).

export class AuthRequiredError extends Error {}
```

Data

JWT + expiresAt live in secure store under moatly.jwt and moatly.jwt-expires-at.

Dependencies

- infra.secure-store (§4.4 of v0.1).

Notes

- 15-minute expiry matches brief §4.2. We refresh at 13 minutes, giving a 2-minute safety margin.
- Refresh is not a sliding-window refresh token. It is a device attestation. Phase 2 uses soft-auth's 'exchange the seed-derived key for a JWT' step; Phase 3 uses the device challenge.

Risks

- Clock skew: a JWT marked 'expires at T' but rejected by the server because the server's T is earlier. Mitigation: refresh on any 401 regardless of local expiry.

Acceptance criteria

- After a fresh setJwt, currentJwt returns it. After app restart, currentJwt still returns it (until expiry).
- ensureFreshJwt within 2 minutes of expiry triggers a refresh, not a stale return.
- clearJwt + currentJwt returns null.

5.3 infra.auth.soft-auth — the time-bounded soft authentication

Responsibility

Authenticate the user in Phase 2 using email + a seed-derived secret. This module exists only until the device ships. §7.3 of this document specifies its removal. The hard expiry (`softAuthExpiresAt`) is a build-time constant set to the planned Phase 3 go-live date.

Public interface

```
export const SOFT_AUTH_EXPIRES_AT: number; // unix ms; build-time constant

export async function softAuthInitiate(
  email: string
): Promise<{ challengeId: string }>;

export async function softAuthComplete(
  challengeId: string,
  emailCode: string,           // 6-digit from the email
  seedProof: Uint8Array       // HKDF('moatly.soft-auth.v1') of master key
): Promise<{ jwt: string; expiresAt: number }>;

export function isSoftAuthExpired(now?: number): boolean;
```

Data

The email address is stored in secure store under `moatly.email-address`. The seed proof is computed per-call; not stored.

Dependencies

- `crypto.keyring` (§4.2 of v0.1) for the master-key-derived proof.
- `infra.api` for the `/v1/auth/*` endpoints.
- `infra.auth.jwt` (§5.2) to store the result.

Notes

Soft-auth is a stand-in. The device is the real auth path.

The Build Brief is explicit that 'a device in some tier is required to enter a Moat' after Phase 3 (§10.4). Soft-auth will be deleted from the codebase, not deprecated-and-left, when the device ships. Do not design features that lean on soft-auth's continued existence.

- On every app start, we check `isSoftAuthExpired()`. If true, and the device path isn't ready yet, we show a blocking screen explaining the outage — not a silent failure.

Risks

- Email delivery to the challenge address can be delayed or spam-filtered. UX must say 'check spam' and give a resend with 30s cooldown.
- Email address can change. Re-auth by email requires proving ownership of the old address. If the user cannot, they must re-onboard from seed phrase — accepted trade-off.

Acceptance criteria

- A test advances time past `SOFT_AUTH_EXPIRES_AT` and confirms soft-auth endpoints throw `SoftAuthDeprecatedError`.
- The email 6-digit code is bound to challengeId on the server — using a code from a different challenge fails.
- No variant of soft-auth accepts email alone. The seed proof is always required.

5.4 infra.api.moat — REST client for Moat endpoints

Responsibility

Typed client for the Moat endpoints listed in §8.3 and §8.5 of the Build Brief. Handles request signing (JWT attach), response parsing via Zod schemas, typed errors. Does not own retry policy — that is the caller's responsibility so the caller can decide whether an operation is idempotent.

Public interface

```
export interface MoatApi {
  join(): Promise<MoatJoinResponse>;
  current(): Promise<MoatState>;
  postMessage(ciphertext: Uint8Array, envelope: OuterEnvelope):
  Promise<PostMessageResponse>;
  sessionReset(): Promise<SessionResetResponse>;

  // Complaints
  fileComplaint(body: ComplaintEnvelope): Promise<ComplaintFiled>;
  listMyComplaints(): Promise<FiledComplaint[]>;

  // WebSocket ticket – short-lived, exchange for upgrade
  wsTicket(): Promise<{ ticket: string; expiresAt: number }>;
}
```

Data

None. Pure transport.

Dependencies

- `infra.auth.jwt` for the Authorization header.
- `Zod` for response schemas.

Notes

- All request bodies that carry ciphertext go as `application/x-megolm-group-message` (per §8.6 of the brief). Other bodies are JSON.
- Every response passes through a Zod schema before reaching the caller. An unexpected field shape is a hard error, not a silent drop.

Risks

- Schema drift between client and server. Mitigated by a shared schemas package that generates both sides from one source.

Acceptance criteria

- 100% of endpoints have a typed response schema. No any types anywhere in `infra.api.moat`.
- A contract test runs the mobile client against a recorded fixture of the backend; drift breaks CI.

5.5 `infra.ws.gateway` — WebSocket client

Responsibility

Maintain a single authenticated WebSocket to `/v1/moats/current/stream`. Handle connection, ticket-based upgrade, heartbeat, reconnection with exponential backoff, and clean teardown on logout or cycle-end.

Public interface

```
export type GatewayStatus = 'disconnected' | 'connecting' | 'connected' | 'failing';

export interface GatewaySubscription { unsubscribe(): void }

export function startGateway(): void;
export function stopGateway(): void;
export function status(): GatewayStatus;

export function onMessage(
  handler: (envelope: InboundEnvelope) => void | Promise<void>
): GatewaySubscription;

export function onStatus(
  handler: (status: GatewayStatus) => void
): GatewaySubscription;
```

Data

In-memory only. The last-delivered message index per Moat is persisted (so we can ask for a backfill after reconnect) but the WebSocket itself keeps no state between sessions.

Dependencies

- Native WebSocket (built into React Native).
- `infra.api.moat` for the ticket exchange.

Notes

- Heartbeat: send a ping every 20 seconds. If no pong within 10 seconds, tear down and reconnect.
- Reconnect backoff: 1s, 2s, 5s, 10s, 30s, 60s, 60s... (capped). Resets to 1s on success.
- On reconnect, we request a backfill from the server via REST for any messages whose index is `> last-delivered`.
- All outbound frames contain at most a ticket and a ping. Messages are POSTed via REST — the WebSocket is delivery-only, by design.

Risks

- iOS aggressively suspends sockets when the app backgrounds. On wake, we treat the socket as torn down and reconnect; we do not rely on a socket surviving backgrounding.

Acceptance criteria

- Reconnection is deterministic and exits the failing state within 60s of network recovery in 1000 simulated network-flap cycles.
- Missed messages during a disconnect are delivered via backfill; end-state on reconnect matches end-state had there been no disconnect.
- No duplicate delivery: the same messageId is emitted at most once to any subscriber per app session.

5.6 domain.moat.session — Megolm session lifecycle

Responsibility

Own the active Megolm session for the user's current Moat. Create the outbound session at cycle start, import inbound sessions from the other members via Olm 1-to-1, rotate on cycle boundary and on any membership change, discard old sessions with explicit zeroisation.

Public interface

```
export async function ensureCurrentSession(): Promise<void>;
// Called on app startup and after reconnect. If no current session or
// the session is for the wrong cycle, create/import as needed.

export async function rotateOnCycleBoundary(): Promise<void>;
// Triggered by domain.cycle.boundary at cycle-end.

export async function rotateOnMembershipChange(
  event: MembershipChangeEvent
): Promise<void>;

export async function currentOutbound(): Promise<OutboundSession>;
export async function findInbound(sessionId: string): Promise<InboundSession | null>;

export async function discardStaleSessions(): Promise<number>;
// Deletes sessions with cycle_number < current AND discarded_at NOT NULL.
// Returns count deleted.
```

Data

megolm_sessions table (§4.5). One outbound and N inbound (one per other member) per cycle.

Dependencies

- crypto.vodozemac (§5.1).
- infra.api.moat for the session-reset endpoint.
- infra.ws.gateway for the key-distribution messages.
- infra.db (§4.3 of v0.1).

Notes

- Session rotation is a three-step dance: (1) server issues reset signal at cycle boundary, (2) clients create fresh outbound sessions and exchange keys via Olm 1-to-1, (3) old sessions are marked discarded and physically deleted 1 hour later (grace for late-arriving messages under the old session).
- A membership change (someone joined or left) forces immediate rotation — post-compromise security per brief §6.2.

Risks

- Key-exchange messages can arrive before the corresponding ciphertext. We maintain an 'orphan ciphertext' queue (bounded, 100 entries, 5-minute TTL) and drain it when a matching key arrives.

Acceptance criteria

- At any time, there is exactly one non-discarded outbound session per `moat_id` per `cycle_number`.
- After a rotation, the previous session's serialized bytes are zeroed before deletion (property test against a deterministic session).
- A 10-member simulated Moat runs 4 cycles, 50 membership changes, and ends with exactly the expected number of sessions, zero leaked ciphertexts.

5.7 domain.moat.messages — on-device inbox

Responsibility

Ingest inbound envelopes from the WebSocket, decrypt via the session module, write to SQLCipher, expose to the UI, and enforce strict chronological order without server-side reordering logic.

Public interface

```
export interface MoatMessage {
  messageId: string;
  moatId: string;
  cycleNumber: number;
  senderPseudonym: string;
  type: 'whisper' | 'major_statement' | 'witness' | 'presence';
  parentMessageId?: string; // for witnesses
  body?: Whisper | MajorStatement | Witness | null; // Presence is body-less
  postedAt: number;
  expiresAt: number;
}

export async function listCurrentMoat(): Promise<MoatMessage[]>;
// Ordered strictly by postedAt ascending.

export async function postWhisper(text: string): Promise<MoatMessage>;
export async function postMajorStatement(m: MajorStatement): Promise<MoatMessage>;
export async function postWitness(parent: string, text: string): Promise<MoatMessage>;
export async function postPresence(): Promise<MoatMessage>;
```

```
export function onNewMessage(
  h: (m: MoatMessage) => void
): () => void;
```

Data

```
CREATE TABLE moat_messages_local (
  message_id      TEXT PRIMARY KEY,
  moat_id         TEXT NOT NULL,
  cycle_number    INTEGER NOT NULL,
  session_id      TEXT NOT NULL,          -- Megolm session used
  message_index   INTEGER NOT NULL,      -- for dedupe
  sender_pseudonym TEXT NOT NULL,
  type           TEXT NOT NULL CHECK (type IN
('whisper', 'major_statement', 'witness', 'presence')),
  parent_message_id TEXT,
  body_plain      TEXT,                  -- decrypted body, 7-day life
  posted_at       INTEGER NOT NULL,
  expires_at      INTEGER NOT NULL,
  deleted_at      INTEGER                -- set by the deletion worker
);
CREATE INDEX idx_messages_posted ON moat_messages_local (moat_id, posted_at);
CREATE INDEX idx_messages_expiry ON moat_messages_local (expires_at) WHERE deleted_at
IS NULL;
CREATE UNIQUE INDEX uq_messages_dedupe ON moat_messages_local (session_id,
message_index);
```

Dependencies

- domain.moat.session for decryption.
- infra.ws.gateway for inbound envelopes.
- infra.api.moat for posting.

Notes

- Ordering is by postedAt, not by arrival time. If a late-arriving message lands between existing messages, the UI must re-render its position correctly — not append.
- The unique dedupe index is on (session_id, message_index). A reply attempt with the same pair is silently ignored (treated as duplicate delivery).
- The body_plain column is intentionally named to make its sensitivity obvious. The deletion worker (§6) wipes this column at expiry.

Risks

- A bug that writes body_plain without the 7-day expires_at is a covenant-level defect. Schema CHECK constraint enforces NOT NULL on expires_at; a unit test enforces expires_at - posted_at == 7*24*3600*1000 at insertion time.

Acceptance criteria

- listCurrentMoat returns messages in strict postedAt-ascending order. Adversarial test inserts 100 randomly-timed messages and verifies order.
- Duplicate delivery from the WebSocket (same message_id) is idempotent at the database level (UNIQUE index).

- No row in `moat_messages_local` has `expires_at` - `posted_at` != 7 days.

5.8 domain.moat.major-statement — six-field structured statement

Responsibility

Own the Major Statement schema defined in §5.4.3 of the Build Brief. Validate input, enforce the 'once per user per cycle' constraint, compose the encrypted body.

Public interface

```
export interface MajorStatementInput {
  role:          string;    // up to 200 chars
  field:         string;    // up to 200 chars
  conflict:      string;    // up to 400 chars
  judge:         string;    // up to 200 chars
  whichQuestion: 'fix' | 'why';
  spiralPosition: 'toward' | 'away' | 'plateau';
  body?:         string;    // up to 2000 chars
}

export function validateMajorStatement(m: MajorStatementInput): ValidationResult;

export async function canPostThisCycle(cycleNumber: number): Promise<boolean>;

export async function composeAndPost(
  m: MajorStatementInput,
  cycleNumber: number
): Promise<MoatMessage>;
```

Data

A single row in a local once-per-cycle guard table:

```
CREATE TABLE major_statement_guard (
  cycle_number    INTEGER PRIMARY KEY,
  posted_at       INTEGER NOT NULL,
  message_id      TEXT NOT NULL
);
```

Dependencies

- `domain.moat.messages` for the posting.

Notes

- The once-per-cycle rule is enforced client-side AND server-side. Client-side is a UX convenience; the server is the source of truth.
- Validation is synchronous and complete. A valid `MajorStatement` never throws in `composeAndPost`.

Risks

- The character limits are a style constraint, not a bandwidth constraint. Respect them — users who hit the 200-char 'role' limit are being nudged toward concision, which is the intended friction.

Acceptance criteria

- `canPostThisCycle` returns false after a successful post; returns true in the next cycle.
- `validateMajorStatement` rejects missing required fields with named errors per field.
- `composeAndPost` writes to `major_statement_guard` in the same transaction as the message row.

5.9 domain.complaints — client-side of the complaint flow

Responsibility

Compose and send a complaint envelope. List the user's own filed complaints (so they can see what they've filed). Display the 'you have been re-seeded' state if the server tells us so. Show nothing about thresholds or other members.

Public interface

```
export type ComplaintReason =
  | 'advice' | 'targeting' | 'commercial'
  | 'off_covenant' | 'abuse';

export async function fileComplaint(
  targetPseudonym: string,
  reason: ComplaintReason,
  referenceMessageId?: string
): Promise<{ complaintId: string; filedAt: number }>;

export async function listMyFiledComplaints(): Promise<FiledComplaint[]>;

export async function acknowledgeReseeding(): Promise<void>;
// called when the UI has shown the user the 'you've been re-seeded' state.
```

Data

Local cache of filed complaint metadata (`complaint_id`, `target_pseudonym`, `reason`, `filed_at`). No content.

Dependencies

- `infra.api.moat` for the file-complaint endpoint.
- `crypto.vodozema` for the envelope encryption.

Notes

The client deliberately knows nothing about thresholds.

Threshold evaluation is server-side only (§5.10.2 of the brief). The client never sees how many complaints exist about a target. It never sees other users' complaints. This is intentional — a client that could see thresholds could be used to weaponise the flow by probing.

- Reassignment, when it happens, arrives as a WebSocket message of type 'reseeded' (from the server). The UI shows a designed screen: 'You have been re-seeded into a new forming Moat. No reason is given. This is by design.'

Risks

- Serial weaponisation detection is server-side. §5.10.3 of the brief says 'a member who files 5+ complaints that do not reach threshold in a rolling 90 days is themselves reviewed.' The client is not told when it is approaching that limit — again, by design.

Acceptance criteria

- The complaint envelope is encrypted under the user's outbound Megolm session before leaving the device. The server receives ciphertext plus metadata only.
- listMyFiledComplaints returns entries in reverse-chronological order.
- No UI path exposes any complaint about any target other than the caller's own filings.

5.10 domain.cycle.boundary — the cycle clock

Responsibility

Display the countdown. Detect cycle-end. Coordinate the cascade: session rotation → deletion pass → rings computation.

Public interface

```
export interface CycleState {
  cycleNumber: number;
  startsAt: number;
  endsAt: number;
  msRemaining: number;
}

export function useCycleState(): CycleState;

export async function refreshFromServer(): Promise<CycleState>;

// Internal to the domain layer – the cycle cascade.
export async function onCycleEnd(): Promise<void>;
  // 1. rotate the Megolm session
  // 2. run the deletion pass
  // 3. recompute the rings for the closed cycle
  // 4. emit 'cycleEnded' to listeners
```

Data

A single 'current cycle' row mirrored from the server:

```
CREATE TABLE cycle_state (
  id          INTEGER PRIMARY KEY CHECK (id = 1),
  cycle_number INTEGER NOT NULL,
  starts_at   INTEGER NOT NULL,
  ends_at     INTEGER NOT NULL
);
```

Dependencies

- domain.moat.session for the rotation call.
- domain.moat.deletion-client for the deletion pass.

- domain.rings for the ring computation.

Notes

- The cascade is a sequence, not a parallel fan-out. Each step must succeed before the next runs. If any step fails the cycle does NOT advance locally — the user sees 'cycle still closing, please wait' until retry succeeds.
- This is intentionally conservative — a partial transition is worse than a delayed one.

Risks

- Cascade wedging — a persistent failure in the deletion pass blocks the cycle indefinitely. Escape hatch: manual 'force-complete cycle' admin action in a debug build, never in production.

Acceptance criteria

- onCycleEnd is atomic-or-retryable. A failure midway through does not leave the app in a half-rotated state.
- A 48-hour CI test simulates 48 cycles and confirms zero wedges.

5.11 ui.moat.* — the Moat home screen and its pieces

Responsibility

Render the Moat in a way that respects every principle in §2 of the brief: strict chronological, no ranking, no ornament, no social features. The Moat home is a feed; below it a composer; above it a cycle countdown.

Components

Component	What it shows
MoatHome	The entire screen — countdown, feed, composer.
CycleCountdown	Days and hours remaining in the current cycle. No urgency, no red, no bell. A gentle greyscale timer.
FeedList	Chronological list of whispers, witnesses threaded under their parents, major statements rendered as cards.
MajorStatementCard	The six-field structured statement. Typography-first; the six fields are visually distinct lines.
WitnessThread	One witness, inline under its parent. A thread has no 'reply-to-reply' — witnesses are flat under the parent.
Composer	Switchable between whisper / major-statement / witness. No reactions. No emoji picker. No attachment button.
PseudonymBadge	Two words, in muted type. Not clickable. Not hoverable. No profile on tap.
PresenceIndicator	A ring of small dots along the top of the feed, one per member who has posted a presence ping in the last 24h.
LampIcon	Persistent, bottom-right. (From Phase 1.)

Notes

What the Moat UI does not do.

No pull-to-refresh (WebSocket pushes). No tap-to-like. No double-tap. No long-press reactions. No haptics on incoming messages. No scroll-to-bottom 'unread count' badge. No sound. No notification dot. The Moat is a quiet place. The UI is what makes it feel quiet.

Acceptance criteria

- No icon, button, or gesture in the Moat home triggers any network request not strictly required for the user's active intent.
- A static analysis gate flags any Moat component that imports from `ui.lamp` or `ui.complaint` inside a modal — those surfaces are separate routes, not embedded within the Moat feed.

5.12 `ui.complaint.*` — complaint filing and review

Responsibility

Two screens. The filing screen lets the user pick a target pseudonym from the current Moat's member list, choose one reason from the closed list, and optionally attach a reference message ID. The review screen shows the user's own filed complaints with their `filed_at` timestamps and reasons.

Components

- `FileComplaintScreen` — target picker, reason radio group, optional message reference, submit with confirm-hold.
- `MyFiledComplaintsScreen` — list with filed-at and reason only.

Notes

- The submit button is a tap-and-hold (1.0s) — similar to the covenant acceptance pattern. Filing a complaint is a deliberate act, not a swipe.
- No 'success' confirmation beyond a single neutral line: 'filed.' No count of how many complaints are near threshold. No celebration of having filed.

Acceptance criteria

- File screen only shows pseudonyms from the user's current Moat's current cycle.
- Review screen cannot be filtered, sorted, or queried beyond its default order.

6. The seven-day deletion client

The Build Brief devotes an entire section (§9) to the seven-day deletion system. Most of §9 is about the server. A version of it also runs on the device, because **a copy of the message lives on the device for its seven-day life**, and we must delete that copy with the same rigour as the server deletes its copy.

Three mechanisms, mirrored from the server:

6.1 Mechanism 1 — row deletion

An hourly worker (scheduled via expo-background-fetch on both platforms, with a foreground fallback for devices where background tasks are unreliable) deletes every row in `moat_messages_local` where `expires_at < now()` AND `deleted_at` IS NULL.

```
-- The client deletion worker loop (simplified)
UPDATE moat_messages_local
SET body_plain = NULL,          -- explicit body wipe first
    deleted_at = unixepoch('now') * 1000
WHERE expires_at < unixepoch('now') * 1000
    AND deleted_at IS NULL
RETURNING message_id;
```

```
-- Then a vacuum to release pages to the OS:
VACUUM;
```

Two steps: **update-and-null-out-body-first**, then VACUUM. This ordering matters — if we rely only on VACUUM, a crash between the DELETE and the VACUUM leaves decryptable content in the DB pages until the next boot.

6.2 Mechanism 2 — Megolm session rotation

Covered in §5.6. Any inbound ciphertext we might still have cached (e.g. in a reconnect-backfill queue) becomes undecryptable once the previous session's inbound state is discarded. This is the cryptographic layer on top of the physical row deletion.

6.3 Mechanism 3 — SQLCipher rekey on the seventh day

A subtler mechanism. Every 7 days we re-derive the SQLCipher database key with a fresh HKDF info parameter (`moatly.sqlcipher.v1:{cycle_number}`). The PRAGMA `rekey` on `op-sqlite` walks every page and rewrites under the new key. If anyone has a forensic image of the device's disk from before the rekey, the pages they have are unreadable under the current key.

This is defence-in-depth against a stolen phone scenario. The threat model (§6.1 of the brief) does not claim to defeat a compromised device, but we raise the cost of the 'acquired old disk image' variant.

6.4 Verification

Three tests run continuously in CI and periodically on real devices:

- **Test 1 — the DB-sweep test:** post 100 messages with `expires_at` in the past, run the worker, confirm the `body_plain` column is NULL for all 100 rows before VACUUM, and that the row count drops to 0 after VACUUM.
- **Test 2 — the page-scrape test:** dump raw bytes of the SQLCipher file after a deletion pass and confirm none of the known test-message plaintext strings appear. (Runs after mechanism 1 + mechanism 3 completes.)
- **Test 3 — the session-forgery test:** attempt to decrypt an old-cycle ciphertext using the current-cycle inbound session and confirm the attempt yields `{ kind: 'wrong-session-id' }`.

6.5 Failure modes

Failure mode	Detection	Response
Background worker did not run for >6 hours.	Timestamp check on app foreground.	Run immediately, surface a quiet banner, report to ops (count only).
VACUUM failed mid-pass.	Exception + retry.	Retry once. Persistent failure pages ops.
PRAGMA rekey failed.	Exception caught by cycle cascade.	Cycle boundary blocks until rekey succeeds. User sees 'cycle still closing.'
User never opens the app — worker never fires.	N/A — expected case.	Delete-on-open. The seven-day promise is held by the server for any user who never opens again; on re-open, the local pass runs before any UI is rendered.

A client that has been offline for 14 days returns to a swept DB.

When the app comes back to foreground after a long absence, the deletion worker runs to completion BEFORE any Moat UI is rendered. The user does not see a 'stale messages are being cleaned up' state. They see nothing, then they see the current cycle. This is non-negotiable.

7. Cross-cutting concerns

7.1 Cycle boundaries — the cascade, end-to-end

When the cycle clock reaches zero, the client performs a strict-ordered cascade. Any failure is fatal to the local transition; the user's Moat remains in the old cycle until retry succeeds.

Cycle rollover (client side)

1. Server broadcasts SessionReset over WebSocket.
2. Client calls `domain.cycle.boundary.onCycleEnd()`, which:
 - a. calls `domain.moat.session.rotateOnCycleBoundary()`
 - create fresh outbound Megolm session
 - exchange session keys with other members via Olm 1-to-1
 - mark previous session discarded (not yet deleted)
 - b. calls `domain.moat.deletion-client.runNow()`
 - sweep `moat_messages_local`
 - VACUUM
 - PRAGMA rekey (mechanism 3)
 - c. calls `domain.rings.recordEvent` for any ring arcs earned in the closed cycle
 - d. deletes discarded Megolm sessions older than 1h
 - e. emits `cycleEnded` event.
3. UI re-renders empty Moat with fresh cycle countdown.

7.2 Offline behaviour

- **Reading** a Moat: works fully offline, up to the seven-day window. Messages are available locally until they expire.
- **Posting** a message offline: queued in an outbox (see below), sent on reconnect.
- **The cycle countdown** continues offline; but the rotation cascade cannot complete until network is restored. Users who are offline through a cycle-end see 'cycle still closing' on reconnect until rotation completes.

7.2.1 The outbox

Outbound messages written while offline sit in an outbox table with a TTL of 24 hours. If a message cannot be delivered in 24 hours, it is dropped and the user is notified once ('1 message could not be delivered — please re-post if still relevant'). This is intentional: queue-forever semantics create strange artefacts when a user comes back online a week later.

```
CREATE TABLE outbox (
  id                INTEGER PRIMARY KEY AUTOINCREMENT,
  moat_id           TEXT NOT NULL,
  envelope          BLOB NOT NULL,          -- prepared ciphertext + outer envelope
  queued_at        INTEGER NOT NULL,
  expires_at        INTEGER NOT NULL,      -- queued_at + 24h
  attempts          INTEGER NOT NULL DEFAULT 0,
  last_attempt_at   INTEGER
);
```

7.3 Soft-auth removal — the Phase 3 cutover

Phase 3 decommissions soft-auth. The following is the playbook for the cutover, for whoever runs it.

- **T-30 days:** send a notice via the email relay to all soft-auth'd users: 'Your device ships in 30 days. Once it arrives, the old email-plus-seed sign-in will stop working. If you lose your seed phrase before then, please re-pair now.'
- **T-14 days:** Phase 3 firmware is in production; the app begins to refuse soft-auth for new JWTs. Existing JWTs still work.
- **T-0:** the app release that deletes `infra.auth.soft-auth` ships. Any user who has not paired a device now sees a 'pair a device (or virtual device) to continue' screen. The virtual device path is always available (brief §5.5.1).
- **T+7 days:** the soft-auth REST endpoints are removed from the server.

The code path leaves the codebase, not just the product.

At T-0, the `infra.auth.soft-auth` module is deleted from the repository — not flag-gated, not commented out. The Git history retains it for audit; the live codebase does not. This is the disciplined-cut pattern that keeps 'temporary' from becoming permanent.

7.4 Error handling, Phase 2 additions

- **Decryption errors on inbound messages:** logged as a shape-only metric ('wrong-session-id': count). Never surfaced to the user. The message is dropped.
- **Rotation failures:** the app displays 'cycle still closing' and retries in background. If failure persists for >30 minutes, we surface a soft error with a 'retry now' button.
- **WebSocket flap:** invisible to the user under 60 seconds. Beyond 60 seconds we show a top-of-screen muted banner 'reconnecting...'.

7.5 Observability — Phase 2 additions

Every new metric in this phase is a non-content shape-only counter. The `emit()` interface from Phase 1 is the only way to send anything to ops. Examples of what we emit:

- `cycle.rotate.succeeded` (counter)
- `cycle.rotate.failed` (counter, with reason enum)
- `deletion.sweep.succeeded` (counter)
- `deletion.sweep.duration_ms` (histogram, no body size leakage)
- `vodozamac.decrypt.error` (counter, keyed by `VodozamacError.kind`)
- `ws.connect.duration_ms` (histogram)
- `outbox.drop` (counter)

8. Data flows

8.1 A user posts a whisper

1. User taps composer, types text, taps 'send'.
2. `ui.moat.composer` → `domain.moat.messages.postWhisper(text)`.
3. `domain.moat.messages`:
 - a. fetches current outbound session from `domain.moat.session`.
 - b. builds inner JSON { type: 'whisper', body: text, v: 1 }.
 - c. serializes → calls `crypto.vodozamac.encryptGroupMessage`.
 - d. constructs outer Protobuf envelope { session_id, message_index, message_type: WHISPER, posted_at_client }.
 - e. POST `/v1/moats/current/messages` with the envelope.
 - f. on 201: insert into `moat_messages_local`, expires_at = posted_at + 7d.
 - g. fire `onNewMessage` so the UI re-renders.
4. The same ciphertext will arrive back via the WebSocket; dedupe by (session_id, message_index) in `moat_messages_local` drops it.

8.2 A user posts a Major Statement

1. User opens composer, switches mode to 'major statement'.
2. UI renders the six-field form.
3. On submit:
 - a. `ui.moat.composer` → `domain.moat.major-statement.composeAndPost()`.
 - b. `validateMajorStatement()` runs synchronously; failures surface inline beside the offending field.
 - c. `canPostThisCycle()` is checked; if false, UX shows 'You have already posted a Major Statement this cycle. Another in 4 days, 2 hours.'
 - d. flow continues as for a whisper, with type: 'major_statement' in the inner JSON.
 - e. on successful post: write to `major_statement_guard`.
4. The server routes the message to the other members; their devices receive the ciphertext and (in Phase 3) light up their glow.

8.3 A member is reassigned to a new Moat

1. Server has determined (via §5.10.2 thresholds) that the user is to be re-seeded at the next cycle boundary.
2. At cycle-end, the server omits the user from the new session's membership list for the old Moat, and issues a 'reseeded' signal over the WebSocket.
3. Client receives 'reseeded':
 - a. `domain.moat.session` discards the old Moat's sessions.
 - b. `moat_messages_local` for the old `moat_id` is wiped (not waiting for expires_at – the user is no longer in that Moat).
 - c. `domain.moat.session.ensureCurrentSession` creates a fresh outbound for the new Moat.
4. UI shows the designed 'You have been re-seeded into a new forming Moat. No reason is given. This is by design.' screen.
5. User dismisses → `acknowledgeRe seeding()`.

8.4 Cycle boundary — end-to-end view

Already specified in §7.1. Worth noting here that every user in a Moat goes through §7.1 within a few seconds of each other, so the session-key exchange step (1a.ii) has a brief flurry of Olm 1-to-1 messages.

9. Test strategy — Phase 2 additions

Phase 1's pyramid (v0.1 §7.1) stands. Phase 2 adds a new category: the adversarial deletion tests.

9.1 Adversarial deletion tests

Five tests that try to break the seven-day deletion promise from five different angles. These are the tests that, if they go green, give us confidence that we have not lied to our users.

Test	What it does
DELETE-1	Post a message with a known magic string. Fast-forward the client clock 8 days. Run the worker. Dump the raw SQLCipher file. Grep for the magic string. Expect zero matches.
DELETE-2	Same as DELETE-1 but with the app backgrounded for the entire 8 days, simulating a user who rarely opens the app. Expect zero matches on next foreground.
DELETE-3	Post 1,000 messages, wait for session rotation, attempt to decrypt the oldest using the current inbound session. Expect 'wrong-session-id' error and zero plaintext recovered.
DELETE-4	Simulate a crash between DELETE-and-VACUUM and reboot. Confirm that body_plain is NULL on the affected rows (the UPDATE must hit the disk before the DELETE).
DELETE-5	Check that after PRAGMA rekey, pages from the old key are unreadable. Implementation: snapshot the DB file before rekey, snapshot after, diff. The diff must be >95% of pages.

9.2 Adversarial session tests

Test	What it does
SESSION-1	Fuzz the ciphertext input to decryptGroupMessage. Confirm every malformed input resolves to 'malformed-ciphertext' or 'decryption-failed' — never crashes.
SESSION-2	Present a ciphertext from session A to session B. Confirm 'wrong-session-id', no partial plaintext leaks.
SESSION-3	Replay an accepted ciphertext. Confirm 'replay' and no UI update.
SESSION-4	Simulate membership change mid-cycle. Confirm forced rotation and that a message sent before the change is NOT readable by the new member.

9.3 Phase 2 exit-gate test additions

The Build Brief's Phase 2 exit gate (§10.3) lists:

- 50-person closed alpha completes 4 cycles with zero data-retention incidents.
- Deletion job success rate $\geq 99.9\%$ over the alpha period.

- One third-party penetration test on the Moat service with no high or critical findings.
- Publication of the first transparency report.

This breakdown proposes three mobile-specific additions, to be ratified with the lead:

- All five DELETE-* tests from §9.1 pass on iOS, Android, and Android-older-API-level CI.
- All four SESSION-* tests pass on both platforms.
- A 72-hour soak test with simulated 10-member Moat activity at alpha-like volume: zero leaked content in the DB dump, zero wedged cycles, memory steady within 20MB.

10. Implementation plan — Phase 2

The Build Brief budgets 8–10 weeks for Phase 2 with the full mobile team (two mobile engineers) plus one backend engineer owning the server counterpart. This plan walks a 10-week path; a 9-week or 8-week path is possible by descoping the complaint-review UI (not the filing UI — the filing is non-negotiable per brief §5.10).

10.1 Weekly plan

Week	Mobile engineer A (crypto + session)	Mobile engineer B (UI + complaint)
1	Rust wrapper crate + UniFFI .udl; CI builds for iOS, Android.	Moat home skeleton; CycleCountdown; PseudonymBadge.
2	crypto.vodozemac TS wrapper; test vectors pass.	FeedList renders stub messages; chronological order test.
3	domain.moat.session; encrypt/decrypt E2E.	Composer switchable between whisper and major-statement.
4	infra.ws.gateway + reconnect logic.	MajorStatementCard rendering; validation messages.
5	domain.moat.messages store; dedupe; ordering.	WitnessThread; inline reply flow.
6	domain.moat.deletion-client + mechanism 1.	Complaint filing UI; tap-and-hold confirm.
7	SQLCipher rekey (mechanism 3); cycle rotate cascade.	MyFiledComplaintsScreen; re-seeded UX.
8	soft-auth client; JWT refresh; /v1/auth integration.	Accessibility pass; dark mode; reduced-motion handling.
9	Adversarial tests DELETE-1..5 and SESSION-1..4.	Soak test harness; 72h run on CI.
10	Bug-burn with alpha dogfood cohort.	Bug-burn with alpha dogfood cohort.

10.2 Dependencies on the backend

- Weeks 2–3: /v1/moats endpoints available on staging (join, current, messages POST, session reset).
- Weeks 3–4: WebSocket /v1/moats/current/stream available on staging.
- Week 6: /v1/moats/current/complaints available on staging.
- Week 8: /v1/auth/* soft-auth endpoints available on staging.

A blocker on any of these shifts the corresponding mobile week by the blocker's duration. The weekly plan assumes the backend breakdown keeps to its own plan.

10.3 Risks to the plan

- **UniFFI bring-up takes longer than one week.** Buffer: one additional mobile engineer for weeks 1–2 (fractional security engineer reallocation).

- **Rotation cascade bugs found during integration.** Buffer: week 9 is deliberately light on feature work for this.
- **Alpha cohort finds a Sev-1 data-retention issue.** No buffer for this — it blocks exit. The mitigation is the test suite in §9, which exists to find these before the alpha does.

11. Open questions

Question	Owner	Needed by	Default if no answer
Confirm the pseudonym word list (and source).	Product + founder	Before Week 1	Curate from an existing public-domain list; 500 evocative nouns, one-word.
Who owns the schema registry for the shared Protobuf files?	Lead + backend lead	Before Week 1	Mobile repo is source-of-truth; backend imports via Git submodule.
What is the retention policy for the outbox if a user is offline for >24h?	Founder + lead	Before Week 7	Drop silently after 24h; notify on next foreground.
Confirm build-time constant <code>SOFT_AUTH_EXPIRES_AT</code> .	Lead	Before Week 8	Phase 3 planned go-live date + 14 days.
Rings computation at cycle rollover — client-side only, or also an opaque server summary?	Lead	Before Week 7	Client only (matches brief §5.6.2).
Complaint filing during cycle transition — allowed?	Product	Before Week 6	Allowed. Complaints target the outgoing cycle until rollover completes.
Re-seeded UX copy (exact wording).	Founder + product	Before Week 7	'You have been re-seeded into a new forming Moat. No reason is given. This is by design.'
Shall the composer auto-save drafts?	Product	Before Week 3	No auto-save in Phase 2. Decide in Phase 3 after dogfood feedback.

12. What Phase 3 will cover

The Phase 3 breakdown picks up from here. The major pieces:

12.1 Device integration

- BLE pairing flow on react-native-ble-plx.
- Capacitive combination capture (Premium tier) and press-pattern capture (Simple tier).
- Virtual device software fallback — in-app simulation of the combination + glow, labelled 'reduced-security'.
- Composite identity code composition and server exchange (brief §5.5.3).

12.2 Glow driver

- The five defined events and their BLE signals.
- Silent rest state — nothing else ever triggers glow.
- Battery-low indication on the device itself; no in-app banner.

12.3 Firmware OTA

- Signed firmware updates initiated from the app.
- Offline signature verification against a pinned key embedded in the app.

12.4 Soft-auth decommission

- Execute the playbook in §7.3. Delete infra.auth.soft-auth. Ship the release.

— *End of Phase 2 Technical Breakdown* —

Moatly · Moat App Phase 2 technical breakdown · April 2026 · v0.1